GUI

프로그램 작성법

저자	임종우 (E-mail : matlab@matlabschool.com)
편집, 배포	임종관 (E-mail : techuser@oxstart.com)

저작권 : 오정 (http:/ / oxstart.com)

배포처 : MATLAB 전문학교 (http:/ / matlabschool.com)



오정 웹호스팅	http://oxstart.com

1. 리눅스와 윈도우서버를 이용한 웹호스팅 서비스와 고객을 위한 웹메일서비스
2. .COM, .NET, .ORG, .CC, .JP, .BIZ, .INFO 도메인 등록서비스
3. 국내외 검색사이트 등록서비스 .. (준비중)
4. 비디오테이프영상을 스트리밍파일등 다양한 형태의 컴퓨터용 파일로 변환하는 서비스
5. 이밖에 다양한 무료서비스

이야기세상	http://ms-book.com

멀티미디어 이야기세상이란 특별한 서비스를 제공하는 곳으로써 다양한 이야기와 다양한 형태의 여러 서비스가 제공되는 곳입니다. 그러나 지금은 준비중이므로 게시판위주로 서비스중입니다.

한글서당에서 제공되던 다양한 강의와 자료를 이곳에서 같이 공개 서비스합니다.
그러나 이전과 같이 강의는 회원에 가입한 사람만 볼 수 있고, 일부 강의는 유료입니다.
1년 구독료가 5천원에서 1만원이고, 한 달에 네 번 공개됩니다. 자세한 내용은 10월 31일 공개됩니다.

MATLAB 전문학교	http://matlabschool.com

임종수의 MATLAB 완벽가이드가 공개된 곳이기도 하며, VHDL, SIMULINK, GUI, ... 등 다양한 강의가 제공되고 있습니다. 동호회형식으로 운영중입니다.
현재 유료 MATLAB 인터넷강의를 준비중입니다.

[그림 1-23]과 같이 listbox에 item을 첨가하였던 것을 기억해보자.
이들 listbox에 첨가된 item들 각각은 문자열이지만, 이들 문자열들은 listbox의 **string** propertyname에 **cell** data type으로 저장된다.
그러므로, listbox에 있는 각각의 item에 접근하기 위해서는 cell의 index를 지정하는 데 사용하는 중괄호(즉, {,})를 다음과 같이 사용해야 한다.
즉,

```
% --------------------------------------------------------------------
function varargout = LsitMarkFace_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.LsitMarkFace.
GUIDemo('listbox1_Callback',gcbo,[],guidata(gcbo))                    ▶ ①
```

①번 line은 삭제하고, 다음과 같은 code를 첨가해 본다.

```
MarkerEdgeColor=get(h,'string');                                      ▶ ②
```

LsitMarkFace_Callback함수의 첫 번째 입력 parameter인 **h**는 listbox자신의 handle을 의미하므로, ②번 line으로부터 listbox의 item들을 모두 MarkerEdgeColor에 저장할 수 있다.

> **주의 :**
> MarkerEdgeColor에는 listbox에 있는 모든 item의 문자열들이 cell data type으로 저장되어 있다는 데 주의하자!

test.m함수의 맨 첫째 줄에 global MarkerEdgeColor이라고 선언하고, ②번 line앞에도 global MarkerEdgeColor이라고 선언한 뒤에, matlab desktop에 있는 command window에서 다음과 같이 실행해 보자.

```
>>test
```

listbox를 click해 본다. 그리고, 다음과 같이 명령을 계속해 본다.

```
>>global MarkerEdgeColor
>> MarkerEdgeColor

MarkerEdgeColor =

    'r'
    'g'
    'b'
    'c'
```

```
    'm'
    'y'
    'k'
    'w'

>> class(MarkerEdgeColor)

ans =

cell
```

listbox에 있는 각각의 item들은 **1, 2, 3**과 같은 방식으로 각각의 item에 자동으로 index가 부여된다.
그러므로,

```
IndexList=get(h,'value');
set(handles.Line_handle,'markerfacecolor',MarkerFaceColor{IndexList});
```

의 code를 첨가해 주면 listbox를 사용하여 markerfacecolor를 바꾸어 줄 수 있다.

이제, test.m함수에 부여한 global MarkerEdgeColor는 모두 지우고, 저장한다.
그리고, test.m함수를 실행하고, 각각의 listbox item을 click할 때마다 marker의 face color가 바뀌는 지 확인해 보자.

> **주의 :**
> listbox와 popupmenu에 포함되어 있는 item들의 index는 "**1**"부터 시작한다는 데 주의하자.

이제, popupmenu component를 사용하는 방법을 알아보자.
사실, checkbox와 radiobutton은 모양만 틀리지, 사용방법은 같다. 마찬가지로, listbox와 popupmenu도 모양만 틀리고 사용방법은 동일하다.
우리는 popupmenu를 사용하여 plotting되는 line에 첨가되는 marker의 모서리(edge) color를 바꾸어 볼 것이다.
우선, popupmenu에 대한 callback함수의 protocol을 test.m함수에 첨가하고, 다음과 같은 code를 callback함수에 추가해 보자.

```
%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%%%%%
function varargout = PopMarkEdge_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.PopMarkEdge.
MarkerEdgeColor=get(h,'string');
IndexPopup=get(h,'value');
set(handles.Line_handle,'MarkerEdgeColor',MarkerEdgeColor{IndexPopup});
%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%%%%%
```

listbox에 첨가한 code와 비교해 볼 때, 단지, marker의 edge color를 제어하기 위해서, markerfacecolor propertyname 대신에 markeredgecolor를 첨가한 것만 다른 것을 알 수 있을 것이다.
이제, 저장하고, test.m함수를 실행해 보자.
우선, marker type에서 Diamond radiobutton을 click하고, popupmenu에서 적당한 marker edge color를 선택한다.
그러면, axes object에 있는 line의 marker edge color가 popupmenu에서 선택한 color로 바뀌는 것을 알 수 있을 것이다.

이제, togglebutton을 제어하는 방법을 살펴보자.
togglebutton은 일반적인 pushbutton과 달리, mouse로 click하면, [그림 1-40(a)]와 같이 버튼이 눌려 진 모양을 가지며, 다시 mouse로 click하면, 버튼이 나온 모양을 갖는 형태의 button이다.



[그림 1-40] Togglebutton의 States.

[그림 1-40(a)]와 같이 togglebutton이 눌린 상태에는 **value**가 "**1**"이고, released state에서는 value가 "**0**"이 된다.
GUIDemo.m함수에서는 togglebutton을 이용하여 Figure Object의 크기를 조절하고 있다.
즉, 현재 figure object의 크기는 [232 269 739 409]인데, togglebutton을 click하면, figure object의 크기가 [232 269 529 409]과 같이 폭(width)이 739에서 529로 작아지도록 만들 것이다. 물론, units는 pixels로 동일하다.
위와 같이 togglebutton을 사용하기 위해서는 먼저 guide함수에 callback함수의 protocol을 test.m함수에 첨가해 주어야 한다.
그리고, 다음과 같이 coding해 주면 될 것이다.

```
%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%%%%%
function varargout = TGML_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.TGML.
% ------------------------------------------------------------------
CurrentFigurePos=get(handles.figure1,'pos');                          ▶ ①
if get(h,'value')                                                     ▶ ②
    set(handles.figure1,'pos',[CurrentFigurePos(1) CurrentFigurePos(2) 529 ...   ▶ ③
    CurrentFigurePos(4)])
    set(h,'string','More>>')                                          ▶ ④
else
    set(handles.figure1,'pos',[CurrentFigurePos(1) CurrentFigurePos(2) 739 ...   ▶ ⑤
    CurrentFigurePos(4)])
    %set(handles.figure1,'pos',CurrentFigurePos)                      ▶ ⑥
    set(h,'string','Less<<')                                          ▶ ⑦
end;
%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%%%%%
```

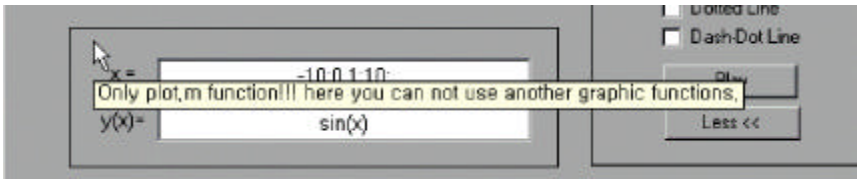
우선, ①번 line과 같이 현재 figure object의 크기를 CurrentFigurePos에 저장한다.
그리고, togglebutton이 눌려 지면, figure의 폭이 작아지고, togglebutton이 release되면, 다시 원래의 figure 크기로 돌아가도록 만들 것이다.
그러므로, togglebutton이 눌린 경우에는 value가 "1"이라고 하였으므로, togglebutton이 눌리면, get(h,'value')는 "1"을 반환하여 참(true)이 된다. 이때에는 figure의 폭만 바꾸어 줄 것이므로, ③번 line과 같이 coding하면 된다. 그리고, togglebutton에 More>>과 같은 문자열을 첨가해 주면 된다.
그러나, 만일, togglebutton이 release 상태라면, get(h,'value')는 "0"을 반환할 것이므로, ⑤번 line과 같이 원래 figure의 크기로 조정해 주면 된다. 그리고, Less<<문자열을 togglebutton에 첨가해 주면 된다.
그러나, 만일, ⑤번 line대신에 ⑥번 line과 같이 coding하면 figure의 원래 크기로 되돌아가지 못한다.
그 이유는 무엇일까?
만일, 독자 중에서 정확한 대답을 할 수 있다면, matlab의 workspace에 대한 개념을 재대로 이해하고 있는 것이다.
이 문제에 대한 대답은 독자 여러분들에게 맡기도록 하겠다.

togglebutton에 대한 coding이 끝났으면, 저장하고, test.m함수를 사용하여 실험해 보자.

지금까지 여러분은 matlab에서 제공되는 모든 Uicontrol component들에 대해서 살펴보았다. 아직, GUIDemo.m함수에는 좀 더 다듬어야 하는 곳들이 많이 남아 있지만, 여러분들이 우선, 알아두어야 할 것은 지금까지의 내용을 한 번 읽어보고, matlab의 GUI programming 기법을 모두 익혔다고 생각하면 안 된다.

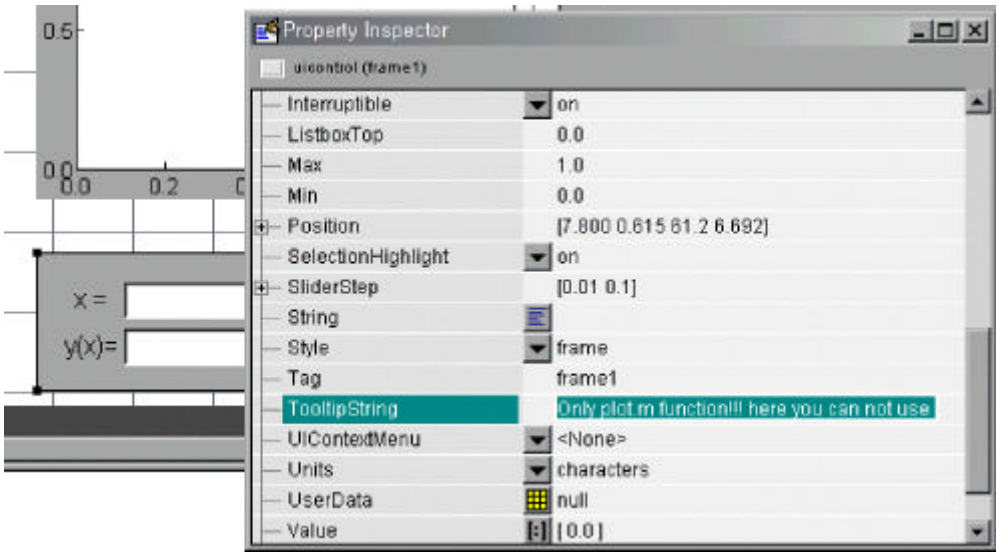
왜냐하면, 완성도 있는 GUI programming을 하기 위해서는 uicontrol, uimenu, uicontextmenu에 대한 각각의 component들과 그에 따른 propertyname/propertyvalue, matlab workspace의 개념, 그리고, 관련 함수들의 유효적 절 한 사용법을 모두 익혀야 하기 때문이다.
결국, 계속해서 matlab으로 GUI program을 작성하면서 coding에 친숙해 지는 것만이 자신 있는 GUI program을 만들 수 있는 길이다. 그리고, 이번에 GUI program을 재대로 배우면, 이후에 SIMULINK를 matlab의 GUI와 연결시킬 때 많은 도움이 될 것이다.
또한, Visual C++로 GUI programming을 할 때 사용하는 handle의 개념과 matlab의 GUI component에서 사용하는 handle의 개념은 같으므로, Visual C++를 배울 때도 많은 도움이 될 것이다.

이제, [그림 1-41]과 같은 Tooltip의 기능을 사용하는 방법을 살펴보자.



[그림 1-41] Tooltip 기능.

tooltip이란, mouse pointer를 일정한 영역에 놓았을 때, [그림 1-41]과 같이 노란색의 네모 상자에 짧은 도움말을 잠시 표시해 주는 기능을 말한다.
tooltip 기능을 사용하는 방법은 의외로 매우 간단하다.
즉, [그림 1-41]은 2개의 editbox를 감싸고 있는 frame component에 mouse pointer가 놓이면, tooltip이 나타나도록 만든 것이다.
이것은 frame component를 [그림 1-42]와 같이 guide함수의 layout editor상에서 선택하고, Property Inspector를 호출하여 tooltip proprtyname에 도움말을 작성해 주면 된다.



[그림 1-42] tooltip 기능을 사용하는 방법.

그런데, 만약, Marker type을 Asterisk로 설정한 경우에는 Marker Face Color가 아무런 의미도 없게 된다.
왜냐하면, Asterisk는 face(임의의 폐공간)가 없기 때문이다.
결국, Asterisk를 선택하면, Marker Face Color를 user가 선택할 수 없도록 ListBox를 비활성화 해 주어야 한다.
이처럼, 임의의 uicontrol, uimenu, uicontextmenu 각각의 item을 비활성화해 주기위해서는 **enable** propertyname을 사용하여야 한다.
enable propertyname에는 다음과 같이 3가지 성질이 있다.

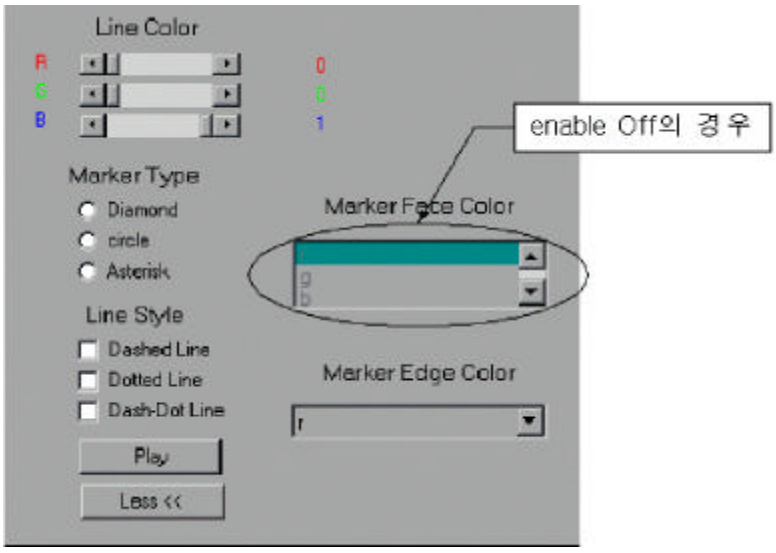
1) On : GUI component가 정상적으로 작동한다.(default)
2) Inactive : GUI component가 정상적인 모습을 가지지만, 작동은 하지 않는다.
3) Off : GUI component가 비활성화 모양(grayed out)을 가지며, 작동하지 않는다.

여기서, 주의 할 것은 enable의 Inactive propertyvalue는 uicontrol component에 대해서만 유효하다.

예를 들면, test.m 함수의 맨 앞에 앞에서와 같이 global handles를 설정하고, test.m함수를 실행해 보자.
이때, listbox의 handle value는 LsitMarkFace이므로 다음과 같이 enable propertyname을 사용할 수 있다.

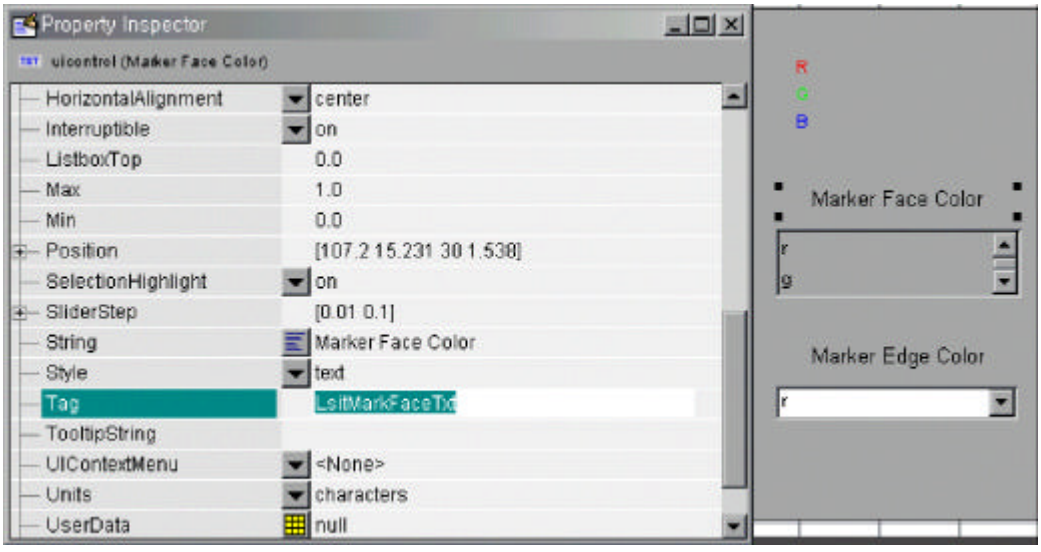
```
>>test
>> global handles
>> set(handles.LsitMarkFace,'enable','off')
```

위와 같이 명령하면, Listbox가 [그림 1-43]과 같이 비활성화 된 것을 볼 수 있다.



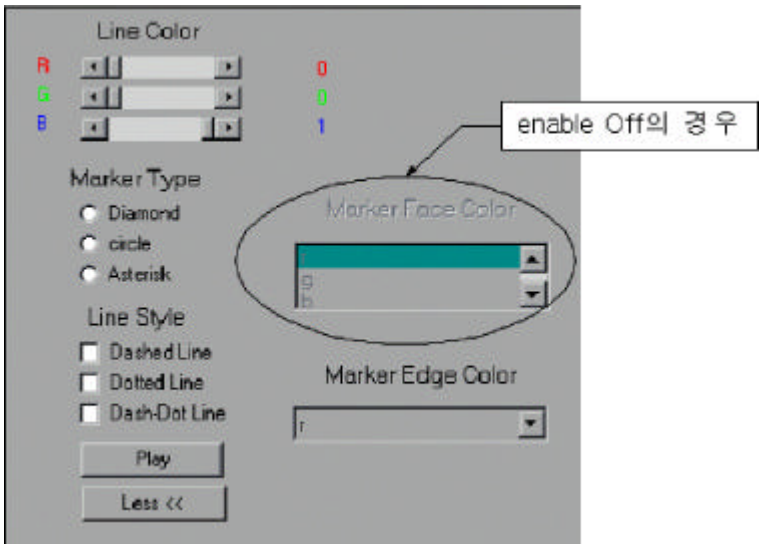
[그림 1-48] **Enable Off의 경우.**

즉, [그림 1-48]과 같이 GUI component가 비활성화되면, mouse로 click하여도 아무런 작동을 하지 않는다.
[그림 1-48]의 경우, listbox의 비활성화가 한눈에 확연히 보이지 않으므로, Listbox의 위에 있는 "Marker Face Color" 문자열도 listbox와 함께 비활성화 되도록 설정해 보자.
[그림 1-49]와 같이 guide 함수를 사용하여 "Marker Face Color" 문자열의 handle value를 LsitMarkFaceTxt으로 설정하고, 저장한다.



[그림 1-49] "Marker Face Color"의 handle value를 LsitMarkFaceTxt으로 설정

다음과 같이 명령하면, [그림 1-50]과 같이 "Marker Face Color" 문자열이 비활성화 된 것을 볼 수 있다.



[그림 1-50] enable off의 효과.

이제, Asterisk radiobutton을 click하면, Listbox와 "Marker Face Color" 문자열이 모두 비활성화 되도록 설정해 보자.
그러기 위해서는 Asterisk의 callback함수인 AMarkRd_Callback에 다음과 같이 listbox를 비활성해 해 주는 routine을 첨가해 주면 된다.

```
set(handles.LsitMarkFace,'enable','off')
set(handles.LsitMarkFaceTxt,'enable','off')
```

이제, test.m함수를 실행 해 보자.

```
>>test
```

그리고, Asterisk radiobutton을 click해 보면, listbox와 "Marker Face Color" 문자열이 모두 비활성화 된 것을 볼 수 있을 것이다.
그러나, 다시 다른 component를 click하면, 계속해서 listbox와 문자열들이 모두 비활성화 된 상태로 남아 있는 것을 볼 수 있다.
그렇다면, 어떻게 해야, 다른 component를 click할 때, listbox와 해당 문자열을 다시 active state로 만들 수 있을 까?
이 문제를 쉽게 해결할 수 있다면, 이제, 여러분은 matlab의 guide함수를 사용한 GUI mechanism을 모두 배웠다고 보아도 된다.

Asterisk radiobutton을 click한 뒤에, 다시, 임의의 다른 component를 click하면, test.m함수가 다시 재귀 호출되며, 이때에는 test('임의의 Callback 함수',gcbo,[],guidata(gcbo))과 같이 입력 매개변수가 사용된다.
그러므로, test.m함수는 다음과 같은 부분을 수행하게 된다.

```
%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%
elseif ischar(varargin{1}) % INVOKE NAMED SUBFUNCTION OR CALLBACK

    try
        [varargout{1:nargout}] = feval(varargin{:}); % FEVAL switchyard ▶ ①
    catch
        disp(lasterr);
    end

end
%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%
```

그러므로, ①번 line에 다음과 같이 code를 첨가해 주면 문제가 해결된다.

```
%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%
elseif ischar(varargin{1}) % INVOKE NAMED SUBFUNCTION OR CALLBACK

    try
        handles=guidata(gcbo);
        if handles.ListBoxOff ==1                                   ▶ ①
           handles.ListBoxOff =0;                                   ▶ ②
           set(handles.LsitMarkFace,'enable','on')
           set(handles.LsitMarkFaceTxt,'enable','on')
           guidata(gcbo,handles);
        end;
        [varargout{1:nargout}] = feval(varargin{:}); % FEVAL switchyard
    catch
        disp(lasterr);
    end
end
%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%
```

①번과 ②번 line에서 사용한 ListBoxOff는 Listbox의 현재 enable 상태를 나타내주는 flag 역활을 수행한다.

즉, 현재 Asterisk radiobutton을 click하였다면, Asterisk의 callback함수는 다음과 같이 ListBoxOff을 handles의 field로 설정하고, "1"을 할당한다.
그리고, guidata.m함수를 사용하여 현재 figure object의 data 영역에 저장한다.

```
% ----------------- test.m (AMarkRd_Callback Subfunction) ------------------
function varargout = AMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.AMarkRd.
Check_hnds=[handles.DMarkRd handles.CMarkRd handles.AMarkRd];
ExChoice(3,Check_hnds)

set(handles.Line_handle,'marker','*')
set(handles.LsitMarkFace,'enable','off')
set(handles.LsitMarkFaceTxt,'enable','off')
handles.ListBoxOff=1;                                                    ▶ ③
guidata(h,handles);                                                      ▶ ④
% ----------------- test.m (AMarkRd_Callback Subfunction) ------------------
```

그리고, 앞에서 언급한 ①번과 ②번 line에서 사용하면 된다.
마지막으로, test.m함수의 초기화 routine에 handles.ListBoxOff=0;을 다음과 같이 첨가 해주면 된다.

```
% ---------------------- test.m (초기화 Routine) --------------------------
if nargin == 0  % LAUNCH GUI
        fig = openfig(mfilename,'new');
        % Use system color scheme for figure:
        set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        % Generate a structure of handles to pass to callbacks, and store it.
        handles = guihandles(fig);
    x_str=get(handles.XEdit,'string');
    y_str=get(handles.YEdit,'string');

    x=str2num(x_str);
    y=eval(y_str);
    handles.Line_handle=feval('plot',x,y);
    CurrentLineColor=get(handles.Line_handle,'color');
    set(handles.Rslider,'value',CurrentLineColor(1));
    set(handles.RVal,'string',num2str(CurrentLineColor(1)));
    set(handles.Gslider,'value',CurrentLineColor(2));
    set(handles.GVal,'string',num2str(CurrentLineColor(2)));
    set(handles.Bslider,'value',CurrentLineColor(3));
```

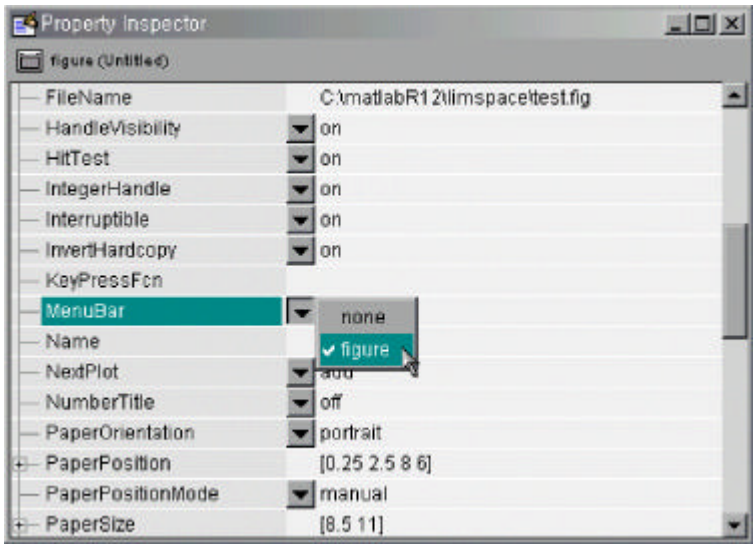
```
    set(handles.BVal,'string',num2str(CurrentLineColor(3)));

    set(handles.TagDiamondMenu,'accelerator','d');
    set(handles.TagcircleMenu,'accelerator','e');
    set(handles.TagAsteriskMenu,'accelerator','k');

    handles.ListBoxOff=0;                                          ▶ ⑤
    guidata(fig, handles);

        if nargout > 0
                varargout{1} = fig;
        end
% ---------------------  test.m (초기화 Routine)  --------------------------
```

지금까지 Uicontrol Object**에 대해서 설명하였다.**
이번에는 uimenu object를 사용하여 test program**에** menu**를 첨가하는 방법에 대해서 알아보자.**
uimenu**를 사용하여** menu**를 첨가하려는 경우, 우선,** matlab**의 기본적으로 제공하는** menus**를 사용하려면, [그림** 1-51**]과 같이 Menubar** propertyname**을 figure로 잡아 주면 된다.**



[그림 1-51] Matlab**이 제공하는 기본** menu.

[그림 1-51]**과 같이** Menubar**를** figure**로 잡은 뒤에, 만일,** matlab**이 제공하는 임의의** menu item**을 지우고 싶다면, 다음과 같이 하면 된다.**

```
>>test
>> global handles
>> handles

handles =

          figure1: 1
       pushbutton1: 247.0013
       BSlidValTxt: 246.0013
       GSlidValTxt: 245.0013
       RSlidValTxt: 219.0015
           text11: 218.0015

          중간   생략

            axes1: 199.0015
      figMenuTools: 154.0018
     figMenuInsert: 142.0018
       figMenuView: 139.0018
       figMenuEdit: 127.0020
  figMenuToolsBFDS: [179.0017  178.0017]
  figMenuCameraPAx: 169.0018

          중간   생략

  figMenuInsertZLabel: 145.0018   ▶ ①
  figMenuInsertYLabel: 144.0018   ▶ ②
  figMenuInsertXLabel: 143.0018   ▶ ③
 figMenuCameraToolbar: 141.0018

          중간   생략

     Line_handle: 248.0005
      ListBoxOff: 0
```

matlab이 기본적으로 제공하는 menubar가 첨가된 test.m함수는 [그림 1-52]와 같다.

7편에서 계속 ...

(이어지는 내용 ☞ Insert menu 중에서 XLabel menu는 삭제하고, YLabel menu의 label을 Y_Label로 바꾸어 보자.)